P2494R1
Relaxing range adaptors to allow for move only types

Published Proposal, 2022-01-17

Author:
Michał Dominiak

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:
SG9, LEWG

Table of Contents

§ 1. Introduction

Currently, many range adaptors require that the user-provided types they store must be copy constructible, which is also required by the assignment wrapper they use, `copyable-box`. This is a consequence of [P1456R1] and [P2325R3] not being integrated together so far. The author believes that this paper is a logical next step from these two papers.

This paper exists, because while writing a paper proposing `views::repeat` (P2474), the author had a desire to make it support move-only types, and needed the tools to specify it. He tried to use the approach in [P2483R0], but has found that approach to not solve the problem correctly.

§ 2. Revision history

§ 2.1. R1

Apply the SG10 recommendation for handling the feature test macro for this change.

Adjust the wording changes to P2474, to align with the wording of a new revision of that paper.

§ 2.2. R0

Initial revision.

§ 3. Design

Similarly to how [P2325R3] turned `semiregular-box` into `copyable-box`, this paper proposes to turn `copyable-box` into `movable-box`. This name is probably not ideal, because it still turns types that happen to be copy constructible into copyable types, but it follows from the prior changes to the wrapper.

The reason why the approach in [P2483R0] is incorrect is that it does not handle types that are copy constructible, but not copy assignable: if a type is copyable, then it works with the wrapper, but if it misses copy assignment, then it does not. The approach taken in this paper solves this issue.

[P2483R0] also suggests future work to relax the requirements on the predicate types stored by standard views. This paper does *not* perform this relaxation, as the copy constructibility requirement is enshrined in the indirect callable concepts ([indirectcallable.indirectinvocable]). Thus, while this paper modifies the views that currently use `copyable-box` for user provided predicates, it only does so to apply the rename of the exposition-only type to `movable-box`; it does not change any of the constraints on those views. It does, however, relax the requirements on invocables accepted by the transform family of views, because those are not constrained using the indirect callable concepts.

In effect, the only views whose constraints are changed by this paper are:

— `single_view`,

— `transform_view`,

— `zip_transform_view`, and

— `adjacent_transform_view`.

Additionally, this paper proposes to modify the changes in P2474 in this same way. If both this paper and P2474 are accepted, we should make `views::repeat` have the same constraints as `views::single`.

§ 3.1. ABI compatibility

The author is not aware of any ABI concerns of this proposal; the question of whether changing constraints on a primary class template has been run by the ABI Review Group, and the responses

indicated that constraints are only mangled in for overloadable templates on the major implementations of C++.

§ 3.2. Feature test macro

After asking for SG10's opinion on the topic on the group's mailing list, this paper proposes to not introduce a new feature test macro for the proposed feature, and rather increment the value of `__cpp_lib_ranges` - which is consistent with how prior changes of similar nature have been handled.

§ 3.3. Implementation experience

The author has implemented the changes to `single_view` and `transform_view` in CMCSTL2. The changes to the box type were not necessary. CMCSTL2 still contains a `semiregular_box`, from before the change to `copyable-box`, but its semiregular box is relaxed to allow for move-only types. Thanks to that, the entire change is replacing "copy" with "move" in the two views mentioned earlier (CMCSTL2 does not include `zip_transform_view` and `adjacent_transform_view`). All the preexisting tests pass with the changes.

§ 4. Wording

§ 4.1. Header `<ranges>` synopsis

Modify Header `<ranges>` synopsis [ranges.syn] as follows:

```
// ...

// [range.single], single view
template<~~copy_constructible~~move_constructible T>
    requires is_object_v<T>
class single_view;

// ...

// [range.transform], transform view
template<input_range V, ~~copy_constructible~~move_constructible F>
requires view<V> && is_object_v<F> &&
         regular_invocable<F&, range_reference_t<V>> &&
```

```
          can-reference<invoke_result_t<F&, range_reference_t<V>>>
class transform_view;

// ...

// [range.zip.transform], zip transform view
template<copy_constructiblemove_constructible F, input_range... Views>
requires (view<Views> && ...) && (sizeof...(Views) > 0) && is_object_v<F> &
            regular_invocable<F&, range_reference_t<Views>...> &&
            can-reference<invoke_result_t<F&, range_reference_t<Views>...>>
class zip_transform_view;

// ...

// [range.adjacent.transform], adjacent transform view
template<forward_range V, copy_constructiblemove_constructible F, size_t N>
  requires see below
class adjacent_transform_view;

// ...
```

§ 4.2. Range factories

§ 4.2.1. Class template `single_view`

Modify Class template `single_view` [range.single.view] as follows:

```
namespace std::ranges {
  template<copy_constructiblemove_constructible T>
    requires is_object_v<T>
  class single_view : public view_interface<single_view<T>> {
  private:
    copyable-boxmovable-box<T> value_; // exposition only (see [range.copym

  public:
    single_view() requires default_initializable = default;
    constexpr explicit single_view(const T& t) requires copy_constructible<
```

```
    constexpr explicit single_view(T&& t);

    // ...
```

```
constexpr explicit single_view(const T& t) requires copy_constructible<T>;
```

11. *Effects*: Initializes `value_` with `t`.

§ 4.2.2. Class template `iota_view`

Do not modify `iota_view`, which explicitly requires `copyable`, and creates copies of the user-provided types as part of its operation.

§ 4.3. Range adaptors

§ 4.3.1. Copyable wrapper

Replace the section Copyable wrapper [range.copy.wrap] with a new section Movable wrapper [range.move.wrap]. The following diff can be applied to the body of [range.copy.wrap] to produce the body of the new [range.move.wrap] section:

1. Many types in this subclause are specified in terms of an exposition-only class template ~~*copyable-box*~~*movable-box*. ~~*copyable-box*~~*movable-box*`<T>` behaves exactly like `optional<T>` with the following differences:
    1. ~~*copyable-box*~~*movable-box*`<T>` constrains its type parameter `T` with ~~`copy_constructible`~~`move_constructible<T> && is_object_v<T>`.
    2. The default constructor of ~~*copyable-box*~~*movable-box*`<T>` is equivalent to:

```
constexpr ~~copyable-box~~movable-box() noexcept(is_nothrow_default_con
    requires default_initializable<T>
  : ~~copyable-box~~movable-box{in_place} {}
```

3. If `copyable<T>` is not modeled, the copy assignment operator is equivalent to:

```
constexpr copyable-boxmovable-box& operator=(const copyable-boxmova
  noexcept(is_nothrow_copy_constructible_v<T>)
  requires copy_constructible<T> {
  if (this != addressof(that)) {
    if (that) emplace(*that);
    else reset();
  }
  return *this;
}
```

4. If `movable<T>` is not modeled, the move assignment operator is equivalent to:

```
constexpr copyable-boxmovable-box& operator=(copyable-boxmovable-bo
  noexcept(is_nothrow_move_constructible_v<T>) {
  if (this != addressof(that)) {
    if (that) emplace(std::move(*that));
    else reset();
  }
  return *this;
}
```

2. *Recommended practice*: ~~copyable-box~~*movable-box*`<T>` should store only a `T` if ~~either `T` models `copyable` or `is_nothrow_move_constructible_v<T>` && `is_nothrow_copy_constructible_v<T>` is `true`~~ :

   1. `copy_constructible<T>` is true, and either `T` models `copyable` or `is_nothrow_move_constructible_v<T>` && `is_nothrow_copy_constructible_v<T>` is `true`, or
   2. either `T` models `movable` or `is_nothrow_move_constructible_v<T>` is `true`.

§ 4.3.2. Class template `filter_view`

Modify Class template `filter_view` [range.filter.view] as follows:

```
namespace std::ranges {
  template<input_range V, indirect_unary_predicate<iterator_t<V>> Pred>
    requires view<V> && is_object_v<Pred>
  class filter_view : public view_interface<filter_view<V, Pred>> {
  private:
    V base_ = V();                                  // exposition only
    copyable-boxmovable-box<Pred> pred_;            // exposition only

    // ...
```

§ 4.3.3. Class template `transform_view`

Modify Class template `transform_view` [range.transform.view] as follows:

```
namespace std::ranges {
  template<input_range V, copy_constructiblemove_constructible F>
    requires view<V> && is_object_v<F> &&
             regular_invocable<F&, range_reference_t<V>> &&
             can-reference<invoke_result_t<F&, range_reference_t<V>>>
  class transform_view : public view_interface<transform_view<V, F>> {
  private:
    // [range.transform.iterator], class template transform_view::iterator
    template<bool> struct iterator;                 // exposition only

    // [range.transform.sentinel], class template transform_view::sentinel
    template<bool> struct sentinel;                 // exposition only

    V base_ = V();                                  // exposition only
    copyable-boxmovable-box<F> fun_;                // exposition only

    // ...
```

§ 4.3.4. Class template `take_while_view`

Modify Class template `take_while_view` [range.take.while.view] as follows:

```
namespace std::ranges {
  template<view V, class Pred>
    requires input_range<V> && is_object_v<Pred> &&
             indirect_unary_predicate<const Pred, iterator_t<V>>
  class take_while_view : public view_interface<take_while_view<V, Pred>> {
    // [range.take.while.sentinel], class template take_while_view::sentine
    template<bool> class sentinel;                      // exposition only

    V base_ = V();                                      // exposition only
    copyable-box<Pred> pred_;                           // exposition only

    // ...
```

§ 4.3.5. Class template `drop_while_view`

```
namespace std::ranges {
  template<view V, class Pred>
    requires input_range<V> && is_object_v<Pred> &&
             indirect_unary_predicate<const Pred, iterator_t<V>>
  class drop_while_view : public view_interface<drop_while_view<V, Pred>> {
  public:
    drop_while_view() requires default_initializable<V> && default_initiali
    constexpr drop_while_view(V base, Pred pred);

    constexpr V base() const& requires copy_constructible<V> { return base_
    constexpr V base() && { return std::move(base_); }

    constexpr const Pred& pred() const;

    constexpr auto begin();

    constexpr auto end() { return ranges::end(base_); }

  private:
    V base_ = V();                                      // exposition only
```

```
    copyable-boxmovable-box<Pred> pred_;                // exposition only

    // ...
```

§ 4.3.6. Class template `zip_transform_view`

Modify Class template `zip_transform_view` [range.zip.transform.view] as follows:

```
namespace std::ranges {
  template<copy-constructiblemove_constructible F, input_range... Views>
    requires (view<Views> && ...) && (sizeof...(Views) > 0) && is_object_v<
             regular_invocable<F&, range_reference_t<Views>...> &&
             can-reference<invoke_result_t<F&, range_reference_t<Views>...
  class zip_transform_view : public view_interface<zip_transform_view<F, Vi
    copyable-boxmovable-box<F> fun_;        // exposition only

    // ...
```

§ 4.3.7. Class template `adjacent_transform_view`

Modify Class template `adjacent_transform_view` [range.adjacent.transform.view] as follows:

```
namespace std::ranges {
  template<forward_range V, copy-constructiblemove_constructible F, size_t
   requires view<V> && (N > 0) && is_object_v<F> &&
            regular_invocable<F&, REPEAT(range_reference_t<V>, N)...> &&
            can-reference<invoke_result_t<F&, REPEAT(range_reference_t<V>,
  class adjacent_transform_view : public view_interface<adjacent_transform_
    copyable-boxmovable-box<F> fun_;            // exposition only

    // ...
```

§ 4.4. Feature test macro

Bump the Ranges feature-test macro in 17.3.2 [version.syn], Header `<version>` synopsis, with the value selected by the editor to reflect the date of adoption of this paper:

```
#define __cpp_lib_ranges 202110L20XXXXL
```

§ 5. Modifications to P2474

> Note: these changes should be applied to the Working Draft if both this paper and P2474 are approved.

Modify Header `<ranges>` synopsis [ranges.syn] as follows:

```
namespace std::ranges {
  // ...

  // [range.repeat], repeat view
  template<copy_constructiblemove_constructible W, semiregular Bound = unre
    requires (is_object_v<W> && same_as<W, remove_cv_t<W>>
      && (is-integer-like<Bound> || same_as<Bound, unreachable_sentinel_t>)
  class repeat_view;

  // ...
}
```

Modify Class template `repeat_view` [range.repeat.view] as follows:

```
namespace std::ranges {
  template<copy_constructiblemove_constructible W, semiregular Bound = unre
    requires (is_object_v<W> && same_as<W, remove_cv_t<W>>
      && (is-integer-like<Bound> || same_as<Bound, unreachable_sentinel_t>)
  class repeat_view : public view_interface<repeat_view<W, Bound>> {
  private:
    // [range.repeat.iterator], class range_view::iterator
    struct iterator;

    copyable_boxmovable-box<W> value_ = W(); // exposition only (see [range
```

```
    Bound bound_ = Bound(); // exposition only

  public:
    repeat_view() requires default_initializable<W> = default;

    constexpr explicit repeat_view(const W & value, Bound bound = Bound())
      requires copy_constructible<W>;

    // ...
```

```
constexpr explicit repeat_view(const W & value, Bound bound = Bound())
  requires copy_constructible<W>;
```

1. *Effects*: Initializes `value_` with `value` and `bound_` with `bound`.

6. Acknowledgements

Thanks to Christopher Di Bella, Corentin Jabot, and Casey Carter for a review and comments of this paper. Thanks to Hui Xie for authoring [P2483R0].

References

Informative References

[P1456R1]
Casey Carter. Move-only views. 12 November 2019. URL: https://wg21.link/p1456r1

[P2325R3]
Barry Revzin. Views should not be required to be default constructible. 14 May 2021. URL: https://wg21.link/p2325r3

[P2483R0]
Hui Xie. Support Non-copyable Types for single_view. 27 October 2021. URL: https://wg21.link/p2483r0